

report_lab0

1.

问题一

没有多人协同开发的经历

问题二

1. 部分提交：方便把多个改动分批次提交。如：两个不相关的改动，可以分在两个提交里，查阅历史更清晰。又如：只改好了一个文件，另一个文件改了一半，可以只提交改好的那一个。
2. 缓冲区：不直接 push 上去，方便撤回操作或补充操作，给自己一个缓冲空间

问题三

git branch 只看本地分支，git branch -a 包括本地分支和远程跟踪分支，远程分支通常以 remotes/ 或 origin/ 开头。-a 是 --all 的缩写，能让人看到本地仓库里记录的所有分支信息，而不仅仅是自己创建的那些。

2.commit

ICS-26Fall-FDU-Noshiro / main.c 



Ju-Song chore: change new line to "Fuck the reality" ✓

8 行 (7 非空行) · 140 Bytes

代码

追溯



源码



```
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Fuck the reality\n");
7      printf("Hello, world!\n");
8  }
```

3.

Commit message

```
$ git commit -m "hello world"
```

```
$ git commit //后可接多行
```

1. 格式化的Commit message的作用。可以提供更多的历史信息，方便快速浏览。可以过滤某些commit（比如文档改动），便于快速查找信息。可以直接从commit生成Change log，方便。

只看行首：

```
$ git log <last tag> HEAD --pretty=format:%s
```

只显示本次新加内容：

```
$ git log <last release> HEAD --grep feature
```

2. Angular 格式：Header 是必需的，Body 和 Footer 可以省略。撤销提交用 revert：开头。

```
<type>(<scope>): <subject>  
// 空一行  
<body>  
// 空一行  
<footer>
```

3. 工具：可用 Commitizen 生成规范提交信息，用 validate-commit-msg 检查提交信息是否符合格式
4. Change log：用 conventional-changelog 自动生成

Gitflow

1. 关键分支

- **master**：与线上环境一致，永久分支，只存放发布版本，每次提交后打 tag。

- **develop**: 开发主分支, 永久分支, 从 master 创建, 功能开发完成后合并回 master。
 - **feature**: 临时分支, 从 develop 创建, 用于开发新功能, 完成后合并回 develop 并删除。
 - **release**: 临时分支, 从 develop 创建, 用于测试和修 Bug, 完成后合并到 master 和 develop 并删除。
 - **hotfix**: 临时分支, 从 master 创建, 用于紧急修复线上 Bug, 修复后合并到 master 和 develop 并删除。
2. **工作流程**: 如 feature → develop → release → master → hotfix。合并时使用 `--no-ff` 保留分支历史。
 3. **版本号规范**: 格式: `0.0.0 ~ 999.999.999`, 最后一个为热修复版本号, 中级位是日常新功能迭代版本号, 首位为重大事件版本号。master 分支上打 tag, 不带 "v" 或者 "V", 只使用数字和".". 分支命名推荐用 `-` 或 `_`, 视团队习惯而定。
-

“为什么要学习 Git”

1. 个人层面:

- 随时回退: 改错了、删错了等等, 都能回到历史版本。
- 实验成本低: 想尝试新方案, 开一个分支, 不影响主线代码; 试错失败直接丢弃。
- 清晰记录: 每次提交都有 commit message, 方便未来查阅理解。

2. 团队层面:

- 并行开发: 不同人可以在不同分支上开发不同功能, 互不干扰。
- 冲突可控: 合并时 Git 能自动合并大部分改动, 冲突也会明确标出, 便于人工解决。
- 责任可追溯: 通过 `git blame`、`git log` 能定位某行代码是谁、何时、为何修改的。

3. 工程层面:

- 代码评审: 进行 Pull Request。
 - Change log 自动生成: 规范化的 commit message 可以直接生成 Change log。
 - 开源和现代交付流程: 掌握现代软件开发的协作方式、版本管理思维和工程规范基础。
-

4. 发生冲突并解决

源代码管理

更改

Merge branch 'feature'

继续

合并更改

main.c

更改

图表

chore: change new line to "..."

chore: add a new line "Life's a struggle"...

Initial commit Noshiro

main.c 3, !

main.c

main()

#include <stdio.h>

int main()

{

// @TODO: print a sentence you want.

采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更

<<<<<< HEAD (当前更改)

printf("qq\n");

printf("Fuck the reality\n");

>>>>>> feature (传入的更改)

printf("Hello, world!\n");

}

在合并编辑器中解决

问题 3

输出

调试控制台

终端

端口

(base) orangesong@Machina-Lucis-Fluentis-10 ICS-26Fall-FDU-Noshiro % git branch

* main

(base) orangesong@Machina-Lucis-Fluentis-10 ICS-26Fall-FDU-Noshiro % git switch feature

Switched to branch 'feature'

(base) orangesong@Machina-Lucis-Fluentis-10 ICS-26Fall-FDU-Noshiro % git switch main

Switched to branch 'main'

Your branch is up to date with 'origin/main'.

(base) orangesong@Machina-Lucis-Fluentis-10 ICS-26Fall-FDU-Noshiro % git checkout main

Already on 'main'

Your branch is up to date with 'origin/main'.

(base) orangesong@Machina-Lucis-Fluentis-10 ICS-26Fall-FDU-Noshiro % git merge feature

Auto-merging main.c

CONFLICT (content): Merge conflict in main.c

Automatic merge failed; fix conflicts and then commit the result.

(base) orangesong@Machina-Lucis-Fluentis-10 ICS-26Fall-FDU-Noshiro %

源代码管理

更改

消息(%Enter 在"main"提交)

提交

图表

Merge branch 'feature' sh...

chore: change new line to "Fuck the re..."

chore: change new line to "qq" shuqiya...

chore: add a new line "Life's a struggle"...

Initial commit Noshiro

main.c

main.c

main()

#include <stdio.h>

int main()

{

// @TODO: print a sentence you want.

printf("Fuck the reality\n");

printf("Hello, world!\n");

}

问题

输出

调试控制台

终端

端口

(base) orangesong@Machina-Lucis-Fluentis-10 ICS-26Fall-FDU-Noshiro % git branch

* main

(base) orangesong@Machina-Lucis-Fluentis-10 ICS-26Fall-FDU-Noshiro % git switch feature

Switched to branch 'feature'

(base) orangesong@Machina-Lucis-Fluentis-10 ICS-26Fall-FDU-Noshiro % git switch main

Switched to branch 'main'

Your branch is up to date with 'origin/main'.

(base) orangesong@Machina-Lucis-Fluentis-10 ICS-26Fall-FDU-Noshiro % git checkout main

Already on 'main'

Your branch is up to date with 'origin/main'.

(base) orangesong@Machina-Lucis-Fluentis-10 ICS-26Fall-FDU-Noshiro % git merge feature

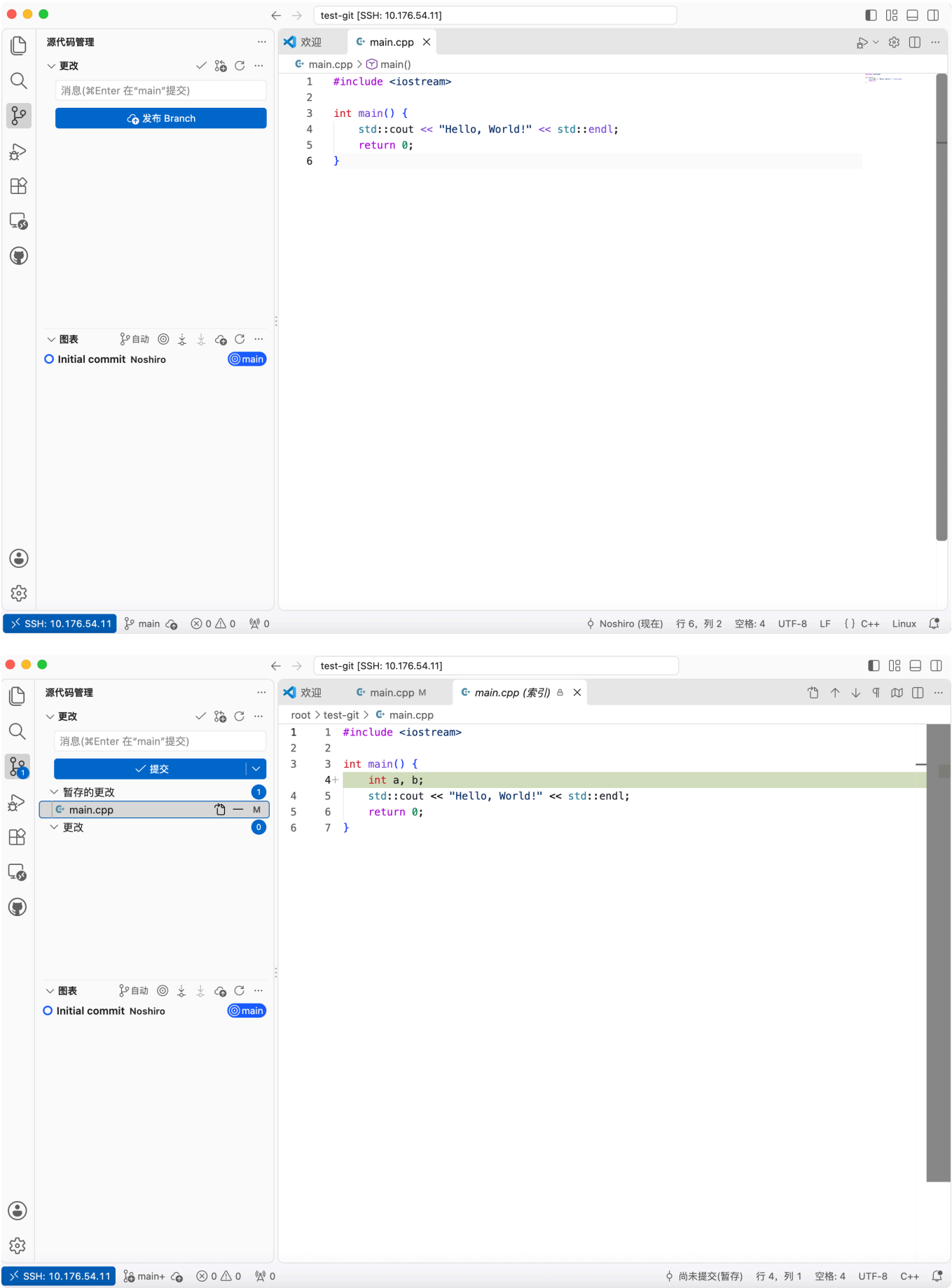
Auto-merging main.c

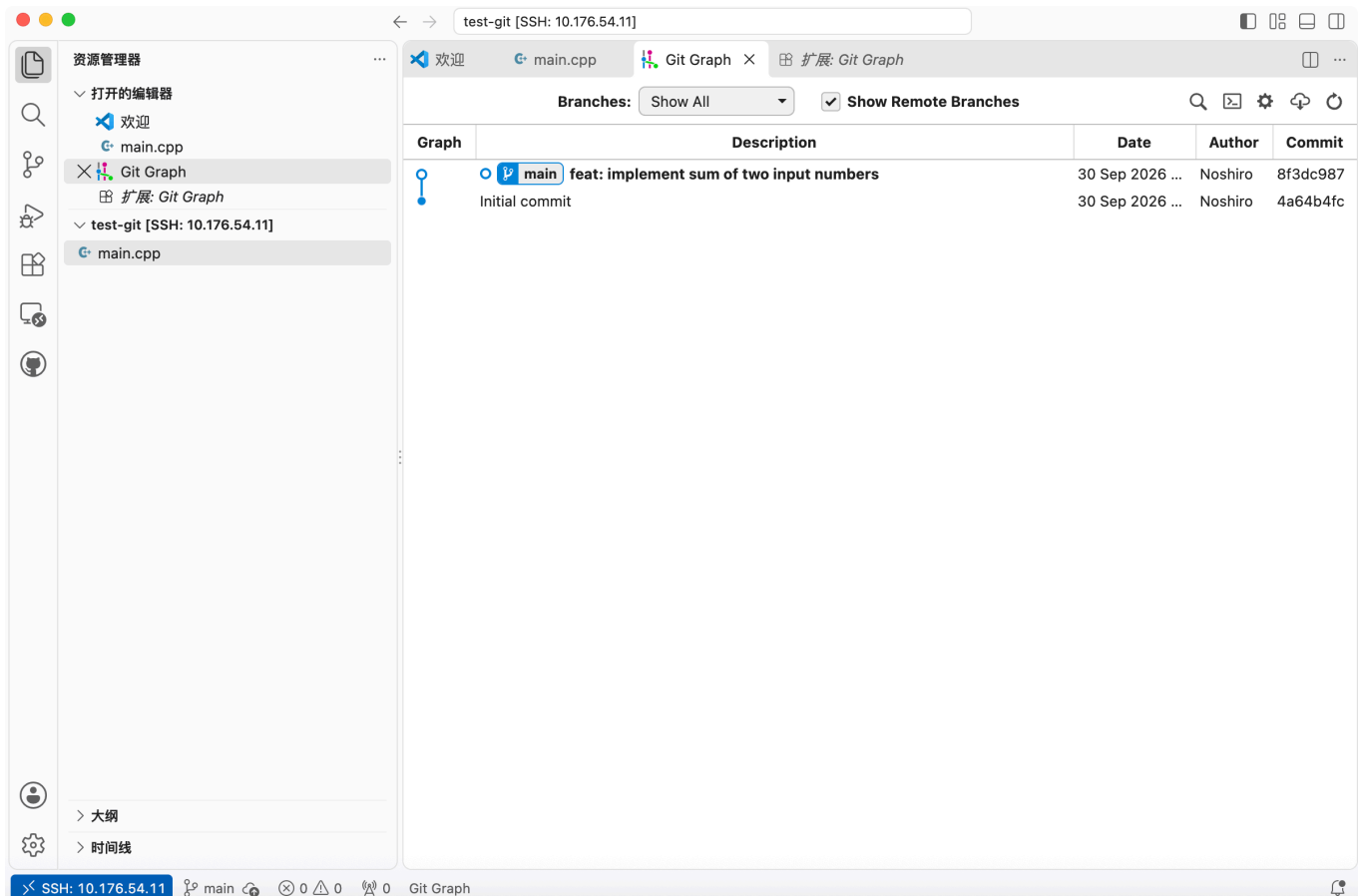
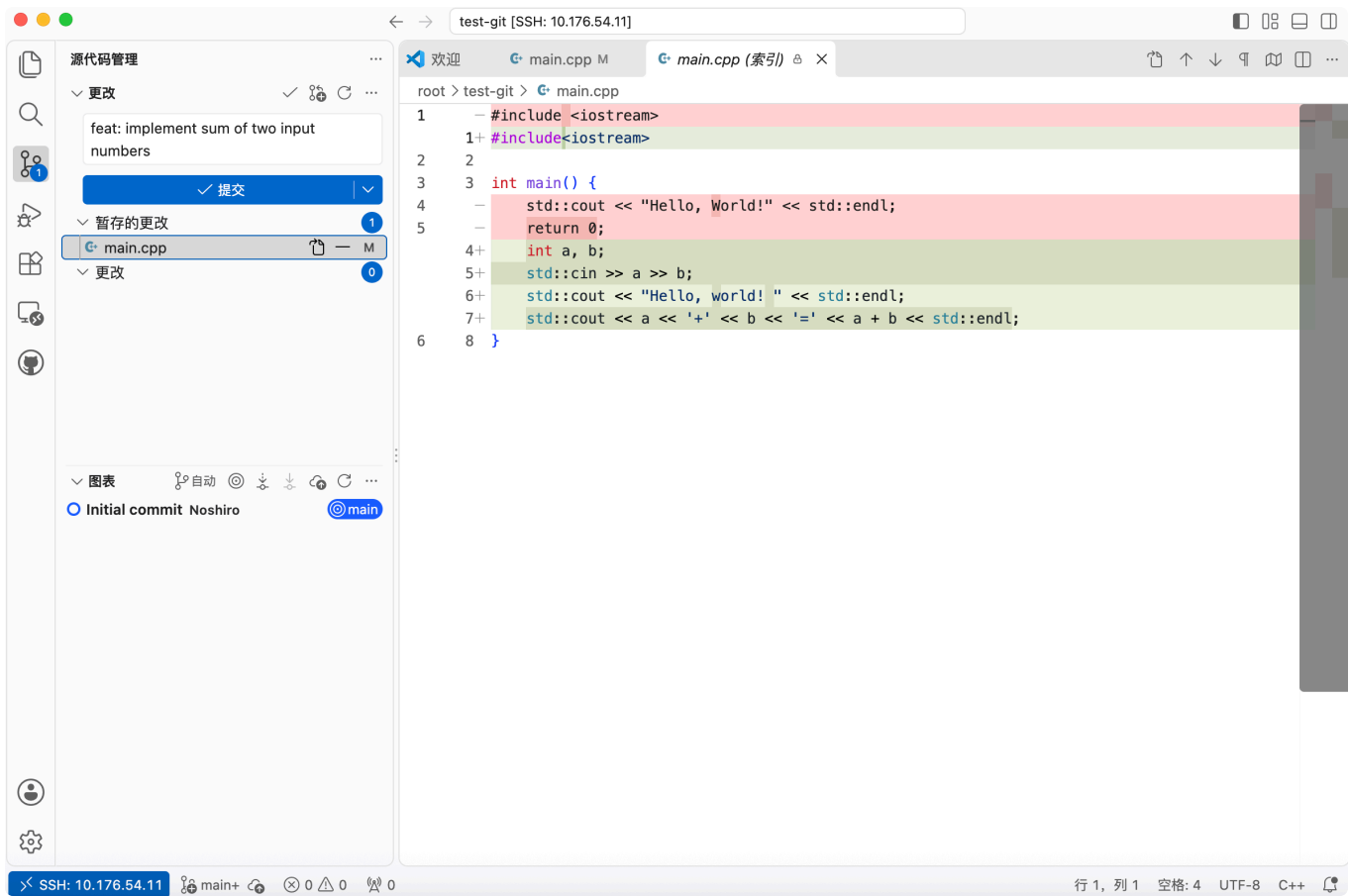
CONFLICT (content): Merge conflict in main.c

Automatic merge failed; fix conflicts and then commit the result.

(base) orangesong@Machina-Lucis-Fluentis-10 ICS-26Fall-FDU-Noshiro %

学习过程截图：



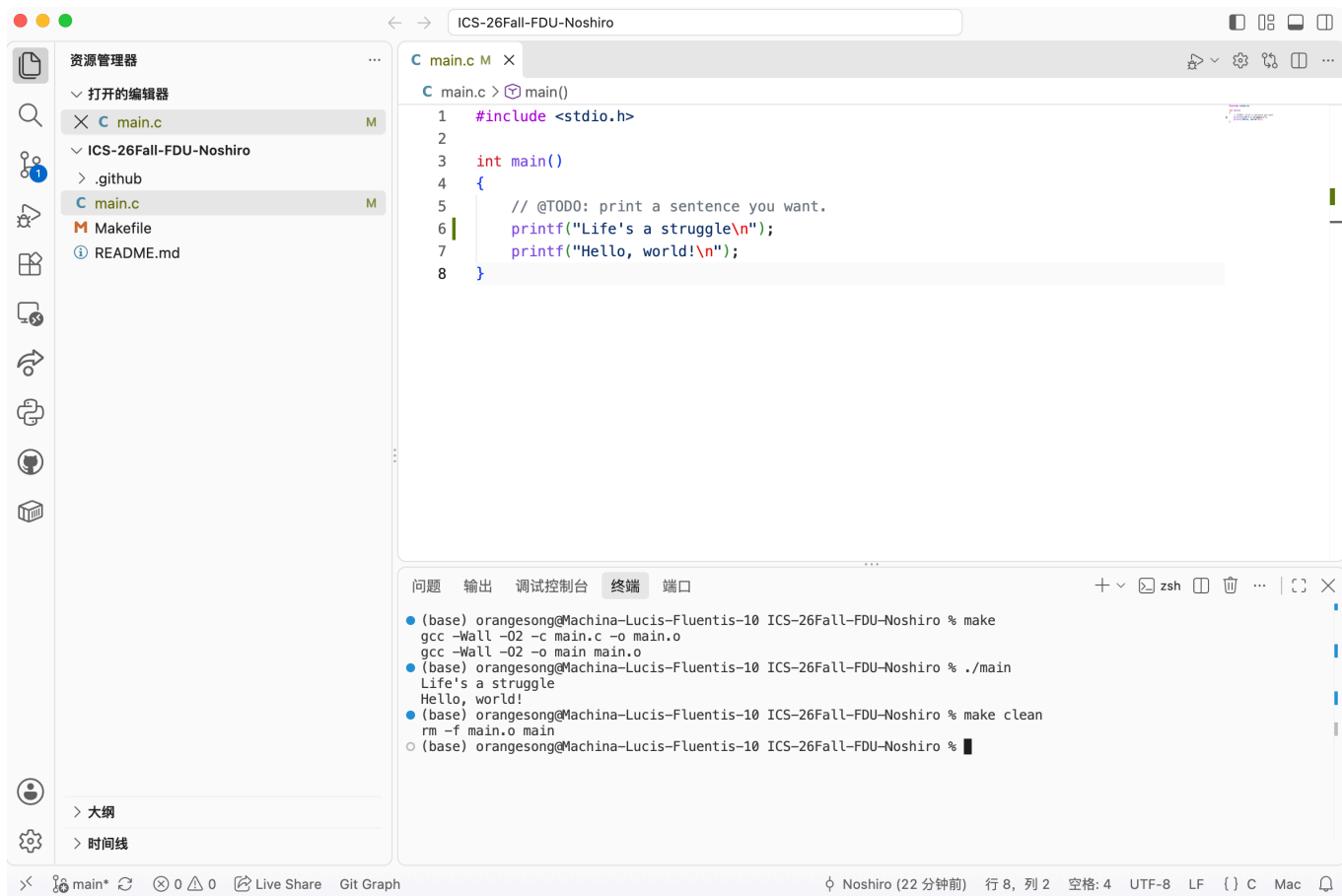




问题 输出 调试控制台 终端 端口

- root@ics:~/test-git# git status
On branch main
nothing to commit, working tree clean
- root@ics:~/test-git# git branch
* main
- root@ics:~/test-git# git branch -a
* main
- root@ics:~/test-git#

```
[(base) orangesong@Machina-Lucis-Fluentis-10 ~ % ssh -T git@github.com
Hi Ju-Song! You've successfully authenticated, but GitHub does not provide shell
access.
(base) orangesong@Machina-Lucis-Fluentis-10 ~ %
```



建议：

要学习的东西太多了，杂七杂八的各种各样的东西太多了，阅读量太大了，又多又难的英文东西一下子映入眼帘，学习曲线太陡峭，而且心里没有底，不知道自己要学到什么程度，一篇文档里往往附有许多其他文档的链接，不知道看到啥时候

实验报告要写的内容也是太多了，而且指令不清晰，在最最后才说到要截图，如果没有一开始通读全文的话在实验的时候可能想不起来要截图，就很麻烦，而一开始就通读全文（相较于直接边看边做）又是非常困难的

感觉可以给一些重要的名词 list，在学习课程之前起码把一些名词的意思搞清楚会好很多，比如“包管理器”

毕竟学校课程应该是帮助人学习而不是直接把人扔进汪洋大海吧